

Forge

Beyond grind

A cooperating proof-search architecture with
conflict learning, certificate arithmetic,
witness synthesis, and induction

Design, algorithms, implementation interfaces,
and reproducible Python experiments

What is delivered. A concrete Lean integration design and three implemented algorithm prototypes: proof-logging CDCL with integer difference logic, exact polynomial cone certificates with a numerical search oracle, and counterexample-guided polynomial invariant synthesis. Saved certificates have a search-independent, standard-library-only Python replay path.

What is not claimed. This bundle is not an implemented Lean tactic. No Lean compiler was available in the execution environment; the Lean examples are uncompiled integration specimens. No experiment compares the prototypes against an executed `grind` invocation.

Prepared for Vladimir Reshetnikov

14 September 2026

Source review targets Lean 4.34.0 and publicly available project documentation as of this date. Third-party package compatibility with that release has not been established.

Abstract

A substantially more capable successor to `grind` should preserve its native, proof-producing treatment of dependent Lean expressions rather than replace it with an untyped external encoding. The proposed `forge` tactic adds a persistent outer proof-search layer and a conflict-learning ground layer around that foundation. Its principal capabilities are demand-driven theorem application and term construction, explicit existential witnesses, certified nonlinear inequalities, induction with parameter generalization, and learning across Boolean alternatives. The useful unit of cooperation is a *scoped proof obligation with a checked explanation*: neither a raw solver verdict nor an unproved conjecture can enter the fact database.

This article specifies the state model, proof boundary, scheduling policy, certificate formats, algorithms, integration points, and evaluation plan. It distinguishes additions to `grind` from facilities already present in Lean’s ecosystem, notably `leanprover/sos`, `leanprover/lp`, `Aesop`, `LeanHammer`, `Lean-SMT`, and the bit-vector SAT infrastructure. Three Python prototypes exercise central mechanisms. On a fixed generated corpus, SAT/difference-logic results agree with exhaustive reference checks in 1,500 cases; all 222 unsatisfiable results have replayed certificates. A restricted polynomial oracle certifies seven of ten examples, while deliberately retaining three unknowns. A synthesis loop discovers and exactly validates six generalized accumulator invariants. These are mechanism tests, not evidence of end-to-end Lean performance superiority.

Contents

1	The design decision	4
1.1	What counts as a significant improvement?	4
2	The actual baseline and available building blocks	4
2.1	Do not design against an obsolete picture of <code>grind</code>	4
2.2	Concrete reuse choices	5
2.3	Capability targets, not untested failure assertions	6
3	A proof boundary that survives aggressive search	6
3.1	The acceptance judgment	6
3.2	Kernel-only and native-checked are different profiles	7
3.3	Scoped facts, not globally valid strings	7
3.4	Conservative arithmetic abstraction	8
4	State model and execution architecture	8
4.1	Three cooperating levels	8
4.2	Core records	8
4.3	Native integration points that were actually inspected	9
4.4	Event flow and scheduling	9
5	Boolean conflict learning with native theory explanations	10
5.1	The additional capability	10

5.2	First-UIP analysis and nonchronological backjumping	10
5.3	A small complete theory island: integer difference logic	11
5.4	RUP replay and the Lean port	12
5.5	Why theory combination needs a contract, not a slogan	12
6	Demand-driven theorem use and term construction	12
6.1	Demand objects extend, rather than rename, backward matching	12
6.2	An implementable retrieval pipeline	13
6.3	Term generation by typed grammar	13
6.4	Quantifier order and inhabitation	13
6.5	Guarded term invention and bounded equality saturation	13
6.6	Instantiation economics	14
7	Nonlinear inequalities through exact certificates	14
7.1	The certificate language	14
7.2	Cheap dictionary search before a full SDP	15
7.3	Why the small oracle must remain explicitly incomplete	15
7.4	Production escalation to SOS	15
7.5	Strict inequalities and denominators	16
7.6	Arithmetic cooperation: bounds and product envelopes	16
7.7	An explicit mixed certificate	16
8	Induction, generalization, and certified lemma synthesis	17
8.1	Where ground automation stops helping	17
8.2	Generalize the parameters that recursive calls change	17
8.3	Counterexample-guided polynomial synthesis	17
8.4	Why the symbolic checks are sufficient	18
8.5	The six discovered invariants	19
8.6	Beyond polynomial accumulators	19
9	Witness synthesis and arithmetic interaction	19
9.1	Witnesses are terms plus obligations	19
9.2	Extracting useful bounds without solving the whole goal	20
9.3	A complete worked cooperation example	20
10	Implementation plan and module boundaries	21
10.1	An incremental route, not a monolithic rewrite	21

10.2 Suggested repository structure	22
10.3 Proposed command surface	22
10.4 Proof size and reconstruction engineering	22
11 Executed experiments	23
11.1 Environment and reproducibility	23
11.2 SAT and integer-difference differential tests	23
11.3 Ablation against the supplied chronological DPLL	24
11.4 Polynomial search and checker results	24
11.5 Invariant synthesis and independent replay	25
11.6 Saved-certificate replay summary	25
11.7 What the experiments establish	25
12 A serious Lean evaluation protocol	26
12.1 Pinning and baseline retention	26
12.2 New-coverage families	26
12.3 Metrics and ablations	26
13 Failure modes and regression obligations	27
14 Reproduction and bundle inventory	28
15 Recommendation	28
A Certificate formats and implementation details	29
A.1 SAT/theory JSON	29
A.2 Polynomial JSON	29
A.3 Invariant JSON	29
A.4 Testing scope and remaining implementation limitations	29
B Lean integration specimens	30

1 The design decision

Build an extension-oriented automation platform around `grind`, not a replacement implementation of everything it already does. Give it an outer layer that constructs useful proof structure and an inner layer that learns from failed combinations of facts. Allow arithmetic and induction engines to produce reusable intermediate lemmas instead of only answering whether the current goal is closed.

A command such as

```
first | grind | aesop | omega | nlinarith | bv_decide
```

is a reasonable baseline, but not the intended design. Each alternative starts too far from what the preceding alternative learned. An arithmetic routine might discover a denominator is positive without closing the goal; that fact should immediately enable a guarded rewrite. An induction search might prove a generalized accumulator lemma; the lemma should become available to a witness constructor and to E-matching. A branch conflict should become a reusable clause with discharged assumptions, not merely a reason to try the next branch.

The goal is *larger useful coverage under controlled resources*. There is no defensible promise of pointwise runtime dominance: adding search can slow easy goals, and dividing a fixed budget between engines necessarily takes resources from somewhere. Forge therefore retains a bounded `grind` lane, measures its overhead separately, and exposes a replay mode which skips discovery once a proof plan has been found.

1.1 What counts as a significant improvement?

A meaningful acceptance criterion has three parts. First, retain a high fraction of the goals that the pinned baseline solves. Second, solve substantial held-out families requiring combinations of induction, construction, nonlinear inequalities, and Boolean search. Third, keep the result independently checkable, with proof reconstruction and kernel checking included in the cost.

The target is not just a larger number of registered lemmas. It is the ability to move between *forms of reasoning*: from a blocked recursive equality to a generalized invariant; from a desired rational expression to a positivity side condition; from an inconsistent assignment to a learned clause; and from a requested existential to a scoped candidate term and its verification conditions.

Status vocabulary used throughout. *Existing* denotes a documented facility in a cited project. *Proposed* denotes Forge architecture not implemented here. *Implemented* denotes the supplied Python code. *Tested* denotes an actually executed experiment whose results are in the bundle. *Specimen* denotes Lean source provided for integration work but not compiled here.

2 The actual baseline and available building blocks

2.1 Do not design against an obsolete picture of `grind`

The Lean reference describes `grind` as a proof-producing combination of congruence closure, propagation, E-matching, structured case analysis, and satellite theory solvers. It also explicitly distinguishes its branching strategy from large Boolean combinatorial solving, for which it recommends `bv_decide`. These are complementary roles, not evidence that `grind` is a weak simplifier. [1]

Its algebraic engine handles polynomial equations through a completion/Gröbner-style procedure; it is not merely a call to normalize both sides of an isolated identity. Separate interfaces provide linear integer and more general linear arithmetic. Reimplementing those engines would spend effort in precisely the area where reuse is most attractive. [2, 3, 4]

The 2026 system description emphasizes native dependent-type congruence closure, typeclass-parametrized solvers, instantiation constraints, and a plugin interface shared with built-in solvers. It already describes conclusion-selected backward patterns, constructors for inductive predicates, interactive control, and invocation minimization. Forge does *not* claim these features as new. Its proposed extensions are a persistent goal-construction layer, genuinely new induction schemas, and a Boolean conflict-learning layer connected to the existing proof-producing machinery. [5]

The release endpoint reported Lean v4.34.0, published on 14 September 2026. Source inspection used this tag for `Main.lean` and `Types.lean`; the online *latest* manual and independently evolving package READMEs are rolling sources. A production manifest must select a mutually compatible set of revisions rather than assume all latest branches work together. [6, 7, 8]

2.2 Concrete reuse choices

Component	Existing building block	Proposed Forge use
Native facts and equalities	Lean’s <code>Lean.Meta.Grind</code> infrastructure	Preserve dependent terms, explanation proofs, and existing solver cooperation.
Outer proof search	Aesop [10]	Reuse rule-indexing and AND/OR search ideas; add induction and demand objects with scoped feedback.
Nonlinear inequalities	<code>leanprover/sos</code> [11]	Call its engine/checker interface after cheaper certificate searches; return intermediate nonnegativity lemmas.
Linear bounds and witnesses	<code>leanprover/lp</code> [12]	Request certified bounds and supported affine witnesses; keep integer reasoning with integer solvers.
SAT and bit-vectors	Lean BVDcide/LRAT infrastructure [14]	Reuse encodings/checkers where appropriate; audit the evaluation trust path instead of blindly composing tactics.
External SMT	Lean-SMT [15]	Optional proof-reconstructing backend on a supported slice, with exact translation obligations.
First-order saturation	Duper and <code>lean-auto</code> [16, 17]	Optional alternative for a retrieved premise slice, not a mandatory conversion of all dependent goals.
Premise selection	LeanHammer [18]	Optional learned ranking over an otherwise deterministic, type-checked retrieval pipeline.
Equality saturation	Existing <code>grind</code> E-graph; research from <code>lean-egg</code> [19, 20]	Extend native demand-guided term generation; do not make the deprecated <code>lean-egg</code> project a production dependency.

Aesop’s normalization, safe, and unsafe phases are a useful starting point, but an allegedly safe rule which commits shared metavariables can remove solutions of sibling goals. Forge must classify safety in terms of the entire shared metavariable state, not just the visual shape of the current target. [10]

The current SOS repository exposes `SOS.Engine.solve`, `SOS.Engine.Result.check`, a Mathlib-free `SOS.Core`, and semantic soundness modules under `SOS.Mathlib`. Its polynomial substrate is `Hex.MvPoly`; earlier material mentioning another substrate is not the current integration contract. The repository already includes rational reconstruction and witness replay. [11]

The current LP project is split into core data, pure verification, tactics, and backend packages. The narrow production boundary should use these slices rather than require native solver loading for every downstream proof. Its documented tactics already include certified maximization and a supported quantified rational-affine fragment; Forge’s novelty is routing and composition, not inventing affine witness solving. [12]

2.3 Capability targets, not untested failure assertions

Goal family	Mechanism to emphasize	Evidence in this bundle
Dependent ground equalities	Reused <code>grind</code> core	Source review only
Large Boolean/theory interaction	CDCL with proof explanations	Python SAT + integer-difference prototype
Polynomial inequalities	Cone certificates, then SOS	Restricted Python search and exact replay
Accumulator identities	Generalized induction + CEGIS	Six exactly checked recurrence invariants
Existential witnesses	Typed term and affine-model search	Proposed algorithms; no general witness implementation
Library-heavy mathematical goals	Goal-directed retrieval and saturation	Proposed integration only
Bit-level verification	Existing verified encoding/checker stack	Proposed integration only

A specimen in one of these families is not automatically a counterexample to `grind`. A suitable library lemma may already make it trivial. A comparative benchmark must hide that shortcut appropriately and execute both systems before labeling a goal as newly solved.

3 A proof boundary that survives aggressive search

3.1 The acceptance judgment

Let Γ be the actual Lean local context and P the target. Forge succeeds only by producing a closed-with-respect-to-metavariables expression p for which the configured checker establishes

$$\Gamma \vdash p : P.$$

The engine may have used floating-point optimization, SAT search, model guessing, or learned ranking to discover p . None of those computations is itself a theorem. Their output must be reduced to a proof term or to a certificate accompanied by a proved soundness theorem and an accepted computation of the checker.

For a certificate family, the desired Lean-facing shape is

```
check : Problem -> Certificate -> Bool
sound : check problem certificate = true -> Denotes problem
```

with a separate, proof-producing reification bridge from `Denotes problem` to the original goal. A theorem about a CNF or a polynomial is insufficient without this bridge. For every abstract atom the bridge records its exact Lean denotation and any required carrier or typeclass evidence.

3.2 Kernel-only and native-checked are different profiles

A verified checker and a verified execution of that checker are different claims. Existing SAT integrations may use compiled reflection; current work such as PBLearn explicitly makes that execution choice. Lean has also evolved how native computations appear in the axiom dependency graph. Therefore Forge must not equate “certificate based” with “no additional evaluation trust.” [24, 25]

The proposed default *strict* profile accepts explicit proof terms or kernel-reducible certificate computations and rejects newly introduced native-evaluation axioms. An optional *native-checked* profile can use a compiled checker, but records that trust choice in the artifact. Both reject `sorryAx`; strict mode also audits transitive axiom dependencies and environment additions, rather than checking for one historically used constant name. Ordinary user-permitted logical axioms, such as classical reasoning, belong to an explicit policy, not an accidental tactic side effect.

For `bv_decide`-style integration, reuse the verified encoding and checker algorithms, but choose the replay path explicitly. In strict mode this may require a proof-producing importer or a kernel-evaluated checker instead of the fastest stock tactic invocation. This is an engineering and proof-size tradeoff, not a reason to silently weaken the contract. The existing `bv_check` interface demonstrates the useful separation of stored SAT certificates from solver installation. [14]

An external solver should preferably run in a subprocess in the strict discovery profile. Mathematical certificate checking does not protect the running Lean process from arbitrary memory corruption in a native FFI. The LP project documents this distinction for its native backend. Forge should serialize requests and responses with explicit limits, validate them, and perform strict replay in a fresh process. [12]

3.3 Scoped facts, not globally valid strings

A proved fact is recorded as

$$F = (P, p, \Delta, S, V, O),$$

where P is its proposition, p its proof, Δ the assumptions it depends on, S its binder/branch scope, V its metavariable and environment version, and O its origin. In production, dependency sets can be compact identifiers rather than copied expressions.

Suppose a branch assumes a and derives b . Caching b as a globally true fact is wrong. Valid exports include the closed implication $a \rightarrow b$, the clause $\neg a \vee b$, or a fact retained only inside the branch. The same discipline applies to theory bounds, E-graph merges, selected witnesses, and lemmas proved using induction hypotheses.

A cache key must account for more than pretty-printed syntax. It includes the expression’s type, relevant typeclass instances, universe instantiation, free-variable closure, declaration/environment identity, transparency policy, and assignments to any relevant metavariables. Closed theorem instantiations can survive more state changes than branch-local normal forms. Store these categories separately.

3.4 Conservative arithmetic abstraction

Lean expressions must not be normalized using the wrong carrier laws. Natural subtraction is truncated; `BitVec` arithmetic wraps; finite-characteristic polynomial reasoning differs from characteristic zero; inversion is totalized in field structures. Clearing a symbolic denominator therefore creates a nonzero or sign obligation rather than erasing it.

Unrecognized real-valued subterms may be treated as independent real atoms for a *sound overapproximation*. A proof valid for every assignment of those atoms is valid for the actual expressions. However, an abstract satisfying assignment can be spurious and is not a counterexample to the Lean goal. Conversely, erasing dependencies in an indexed datatype is not automatically a sound abstraction. Use an explicit bridge or leave that expression in the native layer.

4 State model and execution architecture

4.1 Three cooperating levels

The outer level is an AND/OR graph of proof obligations. An AND node records a refinement whose children must all be proved; an OR node records competing refinements of the same goal. The middle level maintains demands, retrieved lemmas, synthesized candidates, and reusable closed proofs. The inner level performs ground saturation and theory search.

Outer proof construction: introductions, constructors, witnesses, extensionality, generalization, induction	
↓ scoped subgoals	↑ checked proofs and generalized lemmas
Demand manager: target matching, premise selection, candidate terms, guard obligations, resource accounting	
↓ typed facts and candidate obligations	↑ bounds, conflicts, explanations
Ground engine: native E-graph + E-matching + existing arithmetic + CDCL(T) + certificate plugins	

These are logical layers, not three independent copies of every term. Use a shared expression arena and proof DAG, with scope-specific views of facts. Subgoal creation does require separate Lean metavariable contexts when alternatives make incompatible assignments.

4.2 Core records

The following are *proposed interfaces*, not declarations already available in Lean:

```

Fact      = { proposition, proof, assumptions, scope, origin }
Demand    = { target, locals, kind, dependencies, budget, age }
Candidate = { refinement, childDemands, snapshot, estimatedCost }
Conflict  = { premises, certificate, checker, scope }
ProofPlan = { refinements, factDependencies, witnesses, certificates }

PluginResult =
  Closed(proof)
  | Derived(facts)
  | Conflict(explanation)
  | Spawn(demands)
  | Unknown(reason)

```

`Derived` is the crucial addition to a tactic portfolio. A plugin need not solve the current goal to have done useful work. `Unknown` is distinct from both a refutation and an error. A numerical infeasibility report from an incomplete search is normally `Unknown`, unless an independently checked certificate gives it a stronger meaning.

A `Demand` can ask for a theorem, a term of a type, a sign condition, a witness satisfying constraints, a bridge across representations, or a generalized induction statement. The dependency edges explain why resolving one demand would enable another. This is also the information needed for useful failure diagnostics.

4.3 Native integration points that were actually inspected

In the inspected source, `Lean.Meta.Grind.Params` carries extensions, extra theorems, extra facts, simplification data, and configuration. `assertExtra` introduces supplied proof facts through `addNewRawFact`; `mkGoalCore` initializes a goal and solver states. These are concrete adapter points, not a promise that internal APIs are stable. [7]

The inspected types include `GrindM`, `GoalM`, opaque solver extension state, and a `SavedState` containing `Meta` and `grind` state. Its `restore` operation restores both components. Forge needs its own rollback layer for the additional clause database views, demand queues, proof dependencies, and branch-local plugin state. Restoring a Lean metavariable snapshot alone is not enough. [8]

The first implementation should add a small compatibility module for each pinned Lean version. All direct use of internal `Lean.Meta.Grind` entry points lives there. Theory adapters and the outer scheduler target Forge’s records rather than importing dozens of evolving internal modules.

4.4 Event flow and scheduling

A deterministic initial scheduler can use weighted round-robin work queues. For example, assign four small quanta to native propagation, two to demand/theorem work, one to cheap arithmetic, and one to outer search, then admit expensive SOS or external saturation only on stagnation or a high-value demand. These weights are design defaults to tune, not measured optima.

Each queue has an aging mechanism; priority cannot starve an old admissible task forever within an unbounded run. Every operation additionally obeys explicit caps on term count, instantiations, polynomial degree, matrix size, external output size, and generated proof size. Wall-clock deadlines are enforced around subprocesses; native tasks should cooperate with heartbeat and cancellation mechanisms.

```

while budget remains:
  task := chooseAgedTask(queues)
  snapshot := saveRelevantState(task)
  result := runBounded(task)
  match result:
    Closed(p)    => validateAndClose(task.goal, p)
    Derived(fs)  => validateAndPublish(fs)
    Conflict(c)  => checkExplainLearnAndBackjump(c)
    Spawn(ds)    => enqueueWithScope(ds)
    Unknown(why) => recordFrontier(task, why)
  restoreOrCommitAccordingToResult(snapshot, result)

```

This pseudocode deliberately includes validation before publication. To avoid constructing huge unused proofs, explanations may be delayed DAG nodes, but a delayed object has no authority to close a goal until its proof is materialized and checked. Speculative and checked facts must have different types or tags in the implementation.

5 Boolean conflict learning with native theory explanations

5.1 The additional capability

CDCL(T) separates Boolean search from theory consistency while learning explanations from both. Its classical framework is well established; the proposed contribution here is an integration contract with native Lean terms, proof scopes, and the outer demand graph. [21]

Translate the propositional skeleton of $\Gamma \wedge \neg G$ to CNF using definitional variables. Each atom is associated with a Lean proposition, not just a textual formula. A Tseitin variable for $p \wedge q$ carries its defining equivalence and the proof of each generated clause. Classical decidability may be installed where required by the configured logic policy. Boolean abstraction must not assume decidability of arbitrary propositions without the corresponding construction.

Keep terms and typeclass evidence in the native layer. Theory plugins receive assignments to relevant atoms together with proof/explanation handles. They can produce a consequence with its premises, a conflict clause, or a request to split a supported theory condition. No plugin can invent an unproved equality merely to satisfy a guessed model.

5.2 First-UIP analysis and nonchronological backjumping

For each assigned literal store its decision level and its reason clause. At a conflict, resolve the conflict clause against the reasons for the latest assigned current-level literals until exactly one current-level literal remains. The result is an asserting clause. Backjump to the largest decision level among its other literals, add it, and propagate the remaining literal.

```

C := conflictClause
while count(lit in C with level(lit) = currentLevel) > 1:
  v := latest trail variable occurring at currentLevel in C
  C := resolve(C, reason(v), v)
u := unique current-level literal of C
back := max(level(lit) for lit in C except u, default=0)
record certificate for C
backtrack(back)
assert C; enqueue u with reason C

```

For a theory consequence $a_1 \wedge \dots \wedge a_k \Rightarrow b$, the Boolean reason is $\neg a_1 \vee \dots \vee \neg a_k \vee b$. For a theory conflict the final b is absent. All local non-Boolean assumptions used to justify the lemma are retained in its Lean proof environment or discharged in its exported statement.

The initial production version should use two watched literals, stable clause identifiers, occurrence lists, activity-based branching, restarts, and proof-aware clause deletion. The Python prototype intentionally implements none of these engineering optimizations; its scanning propagator makes the central logic inspectable. Lazy theory propagation can be added after reliable conflict explanations, with exact rollback of theory state.

5.3 A small complete theory island: integer difference logic

An atom is $x - y \leq c$, with integer variables and integer constant c . Encode it as an edge $y \rightarrow x$ of weight c . Crucially,

$$\neg(x - y \leq c) \iff y - x \leq -c - 1 \quad (x, y, c \in \mathbb{Z}).$$

The -1 is indispensable and is not valid for real difference logic. Bellman–Ford detects a negative cycle. A checker need not rerun the graph algorithm: it verifies that adding the cited inequalities cancels every variable and yields $0 \leq C$ with $C < 0$.

For example,

$$x - y \leq 0, \quad y - z \leq 0, \quad z - x \leq -1$$

sum to $0 \leq -1$. With atom identifiers 1, 2, 3, a theory certificate can record a signed cycle and the clause $\neg 1 \vee \neg 2 \vee \neg 3$. The checker verifies the signs, integer complements, coefficient balance, and negative total; the order of the cited edges is not logically important once these algebraic conditions hold.

Proposition 5.1. *A successfully checked negative-cycle certificate justifies its recorded theory clause over integer valuations.*

Proof. Assume all signed premises are true. Convert negative literals with the integer-complement equivalence. Add the resulting inequalities. The checker verifies a zero left side and a strictly negative right side, a contradiction. Negating the conjunction yields the clause. $\square$

For a fixed finite set of integer difference atoms and a CNF over them, exhaustive Boolean search plus a complete difference-logic consistency check is a decision procedure. This observation does not extend automatically to all the theories or quantified goals handled by Forge. The prototype exercises only this small island.

5.4 RUP replay and the Lean port

The supplied proof log has two event kinds: independently justified theory clauses and RUP clause additions. A clause C has the reverse-unit-propagation property when unit propagation on the current database together with the negation of every literal in C reaches contradiction. The final event is the empty clause. RUP replay is simpler than reimplementing the searcher’s first-UIP decisions.

The Python replay has a different unit-propagation implementation from the solver and checks theory lemmas by exact summation. It does not trust the solver’s conflict labels. For a Lean port, retain the event sequence and replace its checkers with Lean functions and soundness proofs, or elaborate each event into a shared proof DAG. For larger proofs, hinted LRAT-style replay avoids repeatedly scanning the whole database.

The prototype’s UNSAT result has a certificate; its SAT result has a Boolean assignment and a consistency check. A production theory model can include integer potentials obtained from the shortest-path construction. Such a model is a counterexample only to the *encoded ground problem*; the original Lean goal still requires a faithful model reconstruction and a justified abstraction boundary.

5.5 Why theory combination needs a contract, not a slogan

Do not write “Nelson–Oppen combines everything” in the implementation plan. Finite bit-vectors, integer arithmetic, shared multiplication, and dependent sorts do not jointly satisfy the usual convenient assumptions of a generic disjoint-theory combination theorem. Forge instead exposes explicit bridge lemmas and states completeness separately for each supported combination.

A linear relaxation can reason about a fresh atom w standing for xy , but may only use relationships to x, y which have been proved. A real-valued bound on an integer expression may be imported through a cast theorem; conversely, integrality cuts require integer premises. An equality from the native E-graph must arrive with the exact typed proof which the receiving solver’s reifier understands.

6 Demand-driven theorem use and term construction

6.1 Demand objects extend, rather than rename, backward matching

Conclusion-oriented E-matching already exists in `grind`. [9] Forge adds persistent *proof-construction tasks* which can instantiate a theorem’s conclusion against a desired target, generate missing premises as subgoals, and synthesize arguments not determined by ground matches. It also keeps the resulting work connected to the original target so that irrelevant saturation can be deprioritized.

For a theorem

$$\forall \vec{x}, A_1(\vec{x}) \rightarrow \dots \rightarrow A_k(\vec{x}) \rightarrow B(\vec{x})$$

and demand D , try type-correct unification of $B(\vec{x})$ with D . Arguments fixed by unification are substituted; unresolved arguments become scoped term demands. The instantiated premises become an AND node. The theorem application is assembled only after these demands are solved.

Every attempt occurs in a saved metavariable state. A failed typeclass search or failed premise must not leave assignments behind in a sibling candidate. A fact retrieved under one instantiation is not reusable under a later, incompatible instantiation merely because the theorem name is the same.

6.2 An implementable retrieval pipeline

Build an environment index at module load time. For each declaration store the conclusion head symbols, its binder telescope, relation/type heads, approximate size, and optional trigger information. Start with local hypotheses and explicitly supplied lemmas; then consult a discrimination-tree index for matching conclusions and a symbol-overlap index for potentially useful premises.

An initial deterministic score can be

$$\text{score}(t, D) = 8 \text{ headMatch} + 3 \text{ typeMatch} + 2 \text{ symbolOverlap} + \text{locality} - \text{newPremises} - \frac{1}{4} \text{size}.$$

This is a specified heuristic to benchmark, not an empirically calibrated formula. Retrieve a small batch, try exact elaboration/unification, and expand the batch only if the frontier stagnates. A learned selector such as LeanHammer’s can replace the ranking, but never the type checking or proof reconstruction. [18]

Avoid flooding E-matching with every retrieved theorem. Some declarations are useful as backward rules but disastrous as forward saturation rules. Maintain separate admissibility classes: oriented simplification, ground forward instances, backward target application, constructor/refinement rules, and expensive search-only rules.

6.3 Term generation by typed grammar

For a missing term of type τ , enumerate candidates in increasing cost from local variables, literals justified for τ , constructors, projections, selected applications, and domain-specific templates. A term-production rule records its input types and resulting type; all candidates pass elaboration in their actual local context.

For an affine arithmetic witness, use the LP adapter where its quantified fragment applies. For natural-number witnesses, combine small expressions such as $n + 1$, extrema of known bounds, and integer-solver suggestions. For a structure, construct fields in dependency order. For a function, introduce a lambda and solve its body demand; never choose an arbitrary constant function unless the body obligations justify it.

Examples of target-directed templates include $x + y$, $x - y$, $\max(x, y)$, a boundary point of a certified interval, and a constructor applied to already available subterms. The generator is deliberately finite at each budget. It does not enumerate all expressions of an arbitrary type before trying useful low-cost witnesses.

6.4 Quantifier order and inhabitation

For $\forall x \exists y P(x, y)$, a witness may depend on the introduced x . For $\exists y \forall x P(x, y)$, it may not. Store the allowed free variables in every term demand and check the final witness closure. A guessed Skolem function is not a license to change quantifier order.

Similarly, $\neg \forall x, \neg P(x)$ permits classical existential reasoning, but it does not supply a computable witness algorithm for an arbitrary type. Forge’s constructive witness path first builds an actual term; its classical proof path remains a separate refinement with its own proof and logic policy. Never infer that a type is inhabited because a synthesis template needs a starting element.

6.5 Guarded term invention and bounded equality saturation

Term invention is most useful when tied to a blocked proof. Suppose the target suggests a squared difference $(x - y)^2$ which is absent from the context. Introduce it as a candidate arithmetic atom together with proved expansion or positivity facts. Do not expand all polynomial expressions into every equivalent shape.

Use a budgeted equality-saturation view only for a small target slice. Each rewrite edge has a theorem instance and side-condition proofs. Bound the number of E-nodes, rewrite generations, and extraction candidates; use native typed equality handling. Binder-sensitive rewriting must preserve scope and dependent transports. The research on Lean expressions in E-graphs is directly relevant to these obligations, while the lean-egg repository’s current deprecation makes it a poor default dependency. [20, 19]

An extraction cost should include estimated proof size and side-condition cost, not only expression node count. A syntactically smaller rational expression may be a worse proof representation if it introduces several unproved nonzero denominators.

6.6 Instantiation economics

For multi-pattern matching, select the most restrictive indexed pattern first, join by shared variable bindings, and normalize bindings modulo the current native equivalence relation. Keep an instance cache keyed by theorem, canonical arguments, and relevant scope. Reconsider delayed guards only when a subscribed fact changes, rather than rescanning every delayed instance after every merge.

Existing `grind` instantiation constraints should remain the first control layer. Forge adds demand relevance, downstream success statistics, and conflict participation to scheduling. An instance that appears in many discarded branches should be penalized; one that repeatedly produces small checked conflicts should be promoted. Such statistics influence search only and are excluded from proof artifacts.

7 Nonlinear inequalities through exact certificates

7.1 The certificate language

Let $g_i(\vec{x}) \geq 0$ and $h_j(\vec{x}) = 0$ be assumptions, with rational-coefficient polynomials. To establish $p(\vec{x}) \geq 0$, accept an identity

$$p = \sum_{k=1}^r w_k q_k^2 \prod_{i \in I_k} g_i + \sum_{j=1}^s r_j h_j, \quad w_k \in \mathbb{Q}_{\geq 0}. \quad (1)$$

Indices in I_k may repeat. The equality cofactors r_j have unrestricted signs. Verification checks dimensions, indices, rational coefficients, nonnegative weights, and exact coefficient equality.

Proposition 7.1. *If (1) is checked, then every real valuation satisfying the assumptions satisfies $p \geq 0$.*

Proof. A square is nonnegative, every cited g_i is nonnegative, and each w_k is nonnegative. Every term in the first sum is therefore nonnegative. Each term in the second sum is zero under the equality assumptions. The checked polynomial identity identifies their sum with p . $\square$

This elementary certificate rule is intentionally independent of how its coefficients were found. It can certify a decomposition produced by an LP, an SDP, a symbolic identity search, or a human. The broader sums-of-squares approach to proof reconstruction is established in Harrison’s work; the current Lean SOS library is the practical reuse target. [22, 11]

7.2 Cheap dictionary search before a full SDP

The implemented oracle uses a finite column dictionary. At degree bound D , it includes monomial squares; squares $(m_i \pm m_j)^2$ of pairs of monomials of degree at most $\lfloor D/2 \rfloor$; products of a bounded number of nonnegative assumptions multiplied by monomial squares; and monomial multiples of equalities. The last columns have free coefficients; the other coefficients are constrained nonnegative.

Write these polynomial columns in a common monomial basis to obtain

$$A\lambda = b,$$

with the appropriate sign constraints. A numerical LP proposes λ . Rational reconstruction then tries bounded-denominator approximants, followed, when needed, by an exact linear solve on the proposed support. The candidate is accepted only when the exact checker verifies (1). A tolerance affects discovery, never the mathematical equality test.

The LP objective in the prototype minimizes the sum of nonnegative column weights, leaving equality-cofactor coefficients unpenalized. This is a simple selection heuristic, not a robust sparsity optimizer. A production version can iteratively reweight columns, penalize coefficient bit length, and remove redundant support after a valid certificate has been found.

The number of monomials of total degree at most d in n variables is $\binom{n+d}{d}$. Pairwise-square columns are quadratic in that count, before considering constraints. The prototype enumerates exponent tuples by a simple product-and-filter routine, which is worse than a direct graded monomial generator. Production code should generate compositions directly, exploit sparsity, and enforce caps before allocation.

7.3 Why the small oracle must remain explicitly incomplete

The checker accepts arbitrary polynomial squares, but the discovery dictionary uses only limited square shapes. At degree two it cannot find a certificate for

$$(2x + y)^2 \geq 0$$

from its unconditional monomial and unit-coefficient binomial square columns. To see the obstruction, represent a quadratic form using $x^2, y^2, (x + y)^2, (x - y)^2$. Achieving the cross coefficient $4xy$ requires a difference of binomial weights equal to 2, hence their sum is at least 2. Their contribution to y^2 is then already at least 2, while the target coefficient is 1. Additional nonnegative monomial-square weights cannot fix this.

That failure is not a weakness of sum-of-squares mathematics and not a checker failure. It is a precise limitation of the chosen search dictionary. The supplied test keeps it as an expected Unknown result. A full Gram-matrix SDP, or a richer rational-ratio square dictionary, is the appropriate next stage.

7.4 Production escalation to SOS

The production adapter should submit an eligible reified problem to the existing `SOS.Engine` and use its checked result plus semantic soundness bridge, rather than reimplementing numerical SDP infrastructure. [11] Forge adds the policy around that call: choose the smallest relevant assumptions, cache successful certificate shapes, bound degree and matrix size, and return useful intermediate inequalities as facts.

For a future independent implementation, the usual Gram representation is $p = z^T Q z$ with Q positive semidefinite. Floating-point positive semidefiniteness is not sufficient for acceptance. Recover rational

data, verify a decomposition into nonnegative rational weighted squares, and compare coefficients exactly. Rank-deficient feasible faces are especially troublesome for naive rounding; exact face constraints or reconstruction on a reduced support may be needed. The production decision here is to reuse the current library’s reconstruction facilities, not claim the prototype implements them.

Higher-degree searches and refutation certificates extend coverage, but a fixed-degree cone is not complete for all nonnegative polynomials. A failure to find a certificate is not a counterexample. For an actual negative result, a rational point can suffice when evaluation gives $p < 0$ and all assumptions are independently checked; the prototype does not implement a general counterexample finder.

7.5 Strict inequalities and denominators

A simple strict-positivity route searches for a positive rational ϵ and proves $p - \epsilon \geq 0$. The test example uses $p = x^2 + y^2 + 1$ and $\epsilon = 1/2$. This method is incomplete even for elementary positive functions on open or unbounded domains: positivity need not have a uniform positive gap.

More generally, represent strict facts distinctly and preserve strictness in certificate composition. A contradiction such as $p < 0$ together with a checked proof $p \geq 0$ is valid. Replacing a strict constraint by a weak one may be a sound relaxation for proving a theorem, but can lose exactly the information needed to close it.

For $u/v \geq 0$, demand proofs of the relevant sign conditions. A checked $v > 0$ permits the usual division-order bridge. A mere $v \neq 0$ is sufficient for some equalities but not for preserving the direction of inequalities. Clear rational *constant* denominators by a known positive common multiple; do not treat a symbolic denominator as a positive coefficient.

7.6 Arithmetic cooperation: bounds and product envelopes

A high-value bridge is to ask the linear layer for bounds on variables appearing in nonlinear products. Given

$$\ell_x \leq x \leq u_x, \quad \ell_y \leq y \leq u_y, \quad w = xy,$$

the four exact consequences are

$$\begin{aligned} w &\geq \ell_x y + \ell_y x - \ell_x \ell_y, \\ w &\geq u_x y + u_y x - u_x u_y, \\ w &\leq u_x y + \ell_y x - u_x \ell_y, \\ w &\leq \ell_x y + u_y x - \ell_x u_y. \end{aligned}$$

Each follows by expanding a product of two known nonnegative differences. For example $(x - \ell_x)(y - \ell_y) \geq 0$ gives the first inequality. Thus the nonlinear layer can add linear facts to the LP layer without trusting a floating-point relaxation as a theorem.

Only instantiate envelopes for products relevant to an active demand or a recent conflict. Tighter certified bounds can trigger stronger envelopes. All four bound proofs and the defining equality for w remain dependencies of the resulting facts. If a bound is branch-local, the envelope is branch-local too.

7.7 An explicit mixed certificate

From a real root of a quadratic,

$$at^2 + bt + c = 0,$$

we can derive nonnegativity of the discriminant without assuming $a \neq 0$. The exact identity is

$$b^2 - 4ac = (2at + b)^2 - 4a(at^2 + bt + c). \quad (2)$$

The unrestricted equality cofactor $-4a$ is legitimate: it multiplies a zero assumption, not a nonnegative assumption. This distinction is easy to get wrong in a generic cone representation. A manually supplied certificate for (2) is included and checked; the small search oracle is not credited with discovering it.

8 Induction, generalization, and certified lemma synthesis

8.1 Where ground automation stops helping

Repeatedly unfolding a recursive definition is not a substitute for constructing an induction proof. For a goal about arbitrary input size, the useful new fact is an induction hypothesis applicable to the recursive call. Forge should detect when the blocked goal mentions recursive functions or inductive structures and create an outer induction candidate.

The initial candidate generator reads the recursive argument positions and equation theorems of the relevant definitions. It ranks structural induction variables by how many blocked calls they reduce. For multiple recursive functions, it also considers a common structural argument. It then obtains the appropriate recursor/refinement through Lean’s elaboration machinery, rather than generating unchecked textual proof scripts as the primary interface.

8.2 Generalize the parameters that recursive calls change

Consider an accumulator recurrence

$$F_p(0, a) = a, \quad F_p(n + 1, a) = F_p(n, a + (n + 1)^p), \quad (3)$$

where $n, p \in \mathbb{N}$ and the accumulator is rational. An induction hypothesis specialized to one fixed a does not match the recursive call with accumulator $a + (n + 1)^p$. The right induction statement quantifies over a .

A concrete generalization heuristic constructs a dependency graph of parameters and compares the target’s recursive call with the recursive calls in its defining equations. Parameters occupying positions changed by those calls are candidates for generalization. Revert the selected parameter and every local declaration depending on it; form the resulting dependent telescope in a legal order. Generalization is an OR alternative, not an irreversible preprocessing step, because too much generality can make a useful theorem false or unnecessarily difficult.

For indexed families, index equalities and transported hypotheses may have to be retained in the motive. The tactic must ask Lean to check the motive and minor premises. A textual substitution which ignores dependent hypotheses is not a correct generalization algorithm.

8.3 Counterexample-guided polynomial synthesis

For recurrence (3), search for a polynomial

$$Q(n, a) = \sum_{i+j \leq d} c_{ij} n^i a^j, \quad c_{ij} \in \mathbb{Q},$$

with increasing degree d . Sample executions yield linear constraints on its coefficients. Solve those constraints exactly; when underdetermined, the prototype sets free coefficients to zero. Then check the two symbolic obligations

$$Q(0, a) = a, \quad (4)$$

$$Q(n + 1, a) = Q(n, a + (n + 1)^P). \quad (5)$$

If either residual is nonzero, find a concrete counterexample, add a corresponding sample, and repeat. If the sample constraints are inconsistent at the current degree, raise the degree.

This is a small instance of the synthesis/validation loop commonly called counterexample-guided inductive synthesis. The general idea is established; the contribution here is a concrete exact implementation and a route to proof obligations useful to a Lean tactic. [23]

```

samples := {(0, 0)}
for degree in 0 .. degreeLimit:
  repeat within iterationLimit:
    Q := solveRationalSampleConstraints(samples, degree)
    if inconsistent: break
    residuals := exactBaseAndStepResiduals(Q)
    if residuals are both zero: return certified candidate Q
    sample := concreteCounterexampleFromResidual(residuals, Q)
    samples.insert(sample)
return Unknown

```

The sample evaluator in the prototype uses the finite sum $a + \sum_{k=1}^n k^P$, which directly satisfies the executable recurrence; it does not insert the final power-sum polynomial into the search. Both variables n, a occur in the ansatz. Nevertheless, the recurrence schema and the decision to use a polynomial generalized accumulator are supplied to this experiment. It is not an implemented arbitrary induction-strategy synthesizer.

8.4 Why the symbolic checks are sufficient

Theorem 8.1. *If the polynomial identities (4)–(5) hold, then for every $n \in \mathbb{N}$ and every rational a , $F_P(n, a) = Q(n, a)$.*

Proof. Induct on n , generalizing a . At zero, the recurrence and (4) agree. At $n + 1$, the recurrence gives $F_P(n, a + (n + 1)^P)$. Apply the generalized induction hypothesis at accumulator $a + (n + 1)^P$, obtaining $Q(n, a + (n + 1)^P)$. Identity (5) identifies this with $Q(n + 1, a)$. $\square$

Finite samples only guide discovery. Acceptance rests on exact zero residuals. The supplied independent replay computes those residuals with sparse Fraction-polynomial substitution, without importing SymPy. Thus the synthesis implementation and the replay do not share a symbolic simplifier.

The residual counterexample search also has an exact justification. A nonzero polynomial with degree at most B in each variable cannot vanish on the entire grid $\{0, \dots, B\}^2$: fix one variable and apply the univariate root bound, then repeat for the coefficient polynomials. The prototype chooses a conservative bound exceeding the substitution-induced degrees. A failed step identity at (n, a) implies at least one of its two associated recurrence samples disagrees with Q , because the actual recurrence satisfies that step.

8.5 The six discovered invariants

The experiment synthesized the following forms; every base and step residual was exactly zero:

$$\begin{aligned}
 Q_0(n, a) &= a + n, \\
 Q_1(n, a) &= a + \frac{1}{2}n^2 + \frac{1}{2}n, \\
 Q_2(n, a) &= a + \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n, \\
 Q_3(n, a) &= a + \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2, \\
 Q_4(n, a) &= a + \frac{1}{5}n^5 + \frac{1}{2}n^4 + \frac{1}{3}n^3 - \frac{1}{30}n, \\
 Q_5(n, a) &= a + \frac{1}{6}n^6 + \frac{1}{2}n^5 + \frac{5}{12}n^4 - \frac{1}{12}n^2.
 \end{aligned}$$

These are familiar identities; the experimental point is their discovery from the chosen ansatz and their exact validation, not novelty of the formulas. For a natural- or integer-valued Lean statement, clear known rational denominators and prove the appropriate cast bridge before applying arithmetic normalization.

8.6 Beyond polynomial accumulators

An implementable next layer should offer a few explicit hypothesis languages instead of “ask a model for a lemma.” Examples include affine or polynomial relations between size measures; list equations involving append and accumulator reversal; and relational invariants relating two synchronized recursive calls.

For a reverse-with-accumulator function, a candidate grammar could enumerate equations built from the input list, the accumulator, append, and ordinary reverse. Anti-unifying the blocked induction step with the target often suggests strengthening a specialized statement about an empty accumulator to a statement about arbitrary accumulators. The candidate still needs a complete proof: induction plus simplification and an associativity lemma, retrieved or proved separately.

Guard against circular lemma search. A candidate lemma may use its induction hypotheses only at the recursor-provided smaller arguments. It may not be inserted as an unrestricted fact while trying to prove itself. General theorem dependencies form an acyclic proof DAG, except for recursion represented explicitly by a valid induction or well-founded recursor.

For well-founded induction, enumerate a small set of measures such as length, a numeric parameter, or lexicographic pairs, and demand proofs that recursive calls decrease them. Failure to prove decrease leaves the candidate unresolved; it does not authorize a new termination assumption.

9 Witness synthesis and arithmetic interaction

9.1 Witnesses are terms plus obligations

For $\exists x : \tau, P(x)$, introduce a metavariable $w : \tau$ with the correct permitted dependencies and make $P(w)$ a constraint-bearing demand. Keep alternative witness assignments in separate snapshots. Committing w too early can make later conjuncts impossible; treating each conjunct’s witness as a distinct metavariable can instead prove the wrong statement.

A practical order is: use an existing local term known to satisfy the predicate; try an exact theorem whose conclusion is the existential; invoke a specialized solver on an eligible constraint fragment; enumerate small

typed templates; and finally use bounded synthesis. Every candidate is validated against the original predicate, not just its arithmetic abstraction.

For the goal $\forall n : \mathbb{N}, \exists m : \mathbb{N}, n < m \wedge m \leq n + 2$, the candidate $m = n + 1$ yields elementary integer obligations. For $\exists x : \mathbb{Q}, \forall y : \mathbb{Q}, 0 \leq y \rightarrow y \leq 1 \rightarrow y \leq x$, a certified upper-bound computation can suggest $x = 1$. Quantified affine solving already exists in the LP ecosystem; Forge should delegate its supported fragment and retain a general typed fallback. [13]

9.2 Extracting useful bounds without solving the whole goal

Suppose a nonlinear demand requires $0 \leq x(1 - y) + y(1 - x)$. The relevant linear information is not all hypotheses, but the four facts $x \geq 0, y \geq 0, x \leq 1, y \leq 1$. A backward decomposition of the product nonnegativity proof creates these demands. A bound solver may derive them from more complicated linear assumptions and return them as new facts. The nonlinear checker then produces the target inequality.

This interaction should use exact bound certificates. A numerical optimizer’s decimal optimum is only a candidate bound. A strict open-bound optimum may be unattained; a witness extractor cannot blindly return an optimizer point for every bound certificate. Represent “certified upper bound,” “attained optimum with feasible witness,” and “unbounded” as different outcomes.

9.3 A complete worked cooperation example

Assume real x, y satisfy $0 \leq x, y \leq 1$. Consider

$$0 \leq \frac{x + y - 2xy}{1 + (x - y)^2}.$$

The division bridge asks for a nonnegative numerator and a positive denominator. For the numerator, the cone certificate is

$$x + y - 2xy = x(1 - y) + y(1 - x).$$

For the denominator, the square rule gives $(x - y)^2 \geq 0$, hence $1 + (x - y)^2 > 0$. The checked division-order theorem closes the target. If the four bounds themselves arise from a disjunctive context, the CDCL layer can organize alternatives and learn which combinations are impossible.

No new arithmetic law is involved. The improvement is that the scheduler creates the right demands, obtains their proofs from appropriate components, and preserves their dependency scopes. The standalone polynomial certificate for the numerator is among the seven automatically discovered examples. The entire Lean cooperation pipeline was not executed in this environment.

10 Implementation plan and module boundaries

10.1 An incremental route, not a monolithic rewrite

Stage	Deliverable	Exit test
A	Pin a compatible Lean/Mathlib stack; implement facts, demands, snapshots, and a <code>grind</code> compatibility adapter.	A derived closed fact can be fed into a later native proof; failed alternatives leave no metavariable assignments.
B	Add polynomial reification and a cheap exact certificate consumer, then an SOS adapter.	Mutated certificates are rejected; side-condition dependencies survive rollback; proof artifacts replay without the numerical solver.
C	Add structural induction candidates and changed-parameter generalization.	Custom accumulator and list definitions, without pre-existing result lemmas, are proved and kernel checked.
D	Add scoped witness search and retrieval with separate backward and forward rule policies.	Quantifier-order, dependent-field, empty-type, and shared-witness tests distinguish valid from invalid refinements.
E	Add the Boolean abstraction, theory-explanation interface, and proof-logging CDCL kernel.	Boolean/theory clauses replay in Lean; learned facts do not leak assumptions; actual baseline comparison is available.
F	Add external adapters, certificate minimization, frozen replay plans, and broad benchmarking.	Rebuilds work under the declared trust profile and dependency lock; regressions and new coverage are measured separately.

Stages B–D offer useful capability gains before the most invasive CDCL integration. Stage E may require changes upstream to expose incrementally controllable propagation and explanations efficiently. A conservative first version can call native `grind` on a sliced conjunction of assigned atoms, turn a contradiction proof into a guarded conflict clause, and learn that clause. This is less efficient but establishes the proof contract before fine-grained integration.

The outer induction and witness components should not fork Lean’s elaborator. They should use its existing metavariable, recursor, typeclass, and expression-building facilities. The compatibility module is the only place where internal tactic APIs are directly coupled to the pinned release.

10.2 Suggested repository structure

```
Forge/
  Frontend.lean          -- syntax, configuration, diagnostics
  Core/{Fact,Demand,Scope,ProofPlan}.lean
  Search/{Scheduler,AndOr,Retrieve,Witness,Induction}.lean
  Native/GrindAdapter.lean
  Boolean/{Reify,Trail,CDCL,Explain,Replay}.lean
  Arithmetic/{Reify,Cone,Bounds,Bridge,SOSAdapter,LPAdapter}.lean
  Certificates/{Format,Check,Sound,Audit}.lean
  External/{Protocol,LeanSMT,Duper}.lean
  Replay/{Elaborate,Minimize,Dependencies}.lean
ForgeTest/
  SoundnessRegression/   -- scope, casts, division, witnesses
  NewCoverage/           -- fresh definitions and held-out goals
  BaselineRetention/
  Certificates/          -- valid, mutated, truncated, oversized
```

This structure is a proposal. It is not a description of files implemented in the supplied bundle; the actual bundle inventory appears in Section 14.

10.3 Proposed command surface

```
forge
forge [lemma1, lemma2]
forge (profile := strict) (induction := auto) (nonlinear := true)
forge?                -- emit a compact replay plan
forge replay "proof.forge" -- no discovery or external solvers
```

These commands do not currently exist in the bundle. Their design should preserve a small default surface and move tuning into named profiles. Separate “this engine is disabled” from “this engine was tried and returned unknown.” A failed invocation should report the unresolved demands and the strongest validated intermediate results, not a dump of every internal term.

A replay plan should include used declaration names and instantiations, selected witnesses, induction motives, branch/refinement choices, certificates, and trust-profile metadata. It should not record unstable memory addresses or assume that a pretty-printed term has an unambiguous parse under a future environment. Pin the declarations and verify their types on replay.

10.4 Proof size and reconstruction engineering

Hash-cons repeated expression nodes and share repeated subproofs with let-bound or auxiliary theorem nodes where appropriate. Minimize proof dependencies after a successful run, not on every speculative branch. Track both certificate bytes and elaborated proof-node counts; a compact numerical certificate can still expand into an enormous arithmetic proof.

Use exact polynomial reflection to avoid re-proving every ring operation by thousands of tiny rewriting steps. Conversely, for very small certificates, a direct explicit proof may be smaller and cheaper than reifying a large checker environment. Select the representation by measured size, with both routes obeying the same trust profile.

The clause learner should support deletion for search memory, but the proof log and replay checker must agree on clause lifetimes and identifiers. A clause needed by a stored explanation cannot disappear before reconstruction unless its derivation is retained or the proof is rewritten accordingly. The prototype avoids this issue by never deleting clauses.

11 Executed experiments

11.1 Environment and reproducibility

The supplied experiments ran under Python 3.13.5, NumPy 2.3.5, SciPy 1.17.0, and SymPy 1.14.0 on Linux x86-64. Random seeds are fixed, with main seed 20260914. These are versions observed in this execution, not a claim that they are the latest releases. The exact environment and timings are stored in `results/summary.json`.

The SAT solver and all saved-certificate replay paths use only Python’s standard library. SciPy/HiGHS supplies the polynomial search LP, and SymPy supplies exact search-side linear algebra and invariant synthesis. Running the replay with `python -S` passed, demonstrating that no site-installed search package is required for the stored corpus.

No Lean installation was available, and a Lean toolchain was not obtained. Thus none of the Lean specimens was elaborated, no `grind` benchmark was executed, and no Python replay should be described as Lean kernel verification. This limitation determines the scope of the conclusions below.

11.2 SAT and integer-difference differential tests

The generated corpus contains 1,000 pure CNFs and 500 CNFs with integer-difference atoms. Each formula has between three and eight Boolean variables; clauses contain three distinct variables with independently chosen signs. The number of clauses ranges from one to six times the Boolean variable count. Difference constraints use four integer variables and constants from -3 through 3 .

The reference result enumerates all Boolean assignments and, for theory cases, uses Floyd–Warshall consistency checks. The tested solver instead uses CDCL with Bellman–Ford theory conflicts. These are small exhaustive tests, not a large industrial benchmark.

Check	Result
Random pure CNF cases	1,000
Random CNF + integer-difference cases	500
Satisfiable results	1,278
Unsatisfiable results	222
Differential mismatches	0
UNSAT certificates independently replayed	222
Truncated certificates rejected	222
Additional structural edge cases	5
Deliberately mutated theory certificates rejected	2

The edge cases cover an empty conjunction, an empty clause, a tautology, contradictory units, and repeated literals. SAT assignments are checked against all original clauses and the independent theory oracle. The

integer-complement boundary has a separate regression assertion. A certificate corpus retains the worked negative cycle; random certificates are replayed during the test run rather than all saved as separate files.

11.3 Ablation against the supplied chronological DPLL

The comparison baseline is the bundle’s own chronological, unit-propagating DPLL implementation, with the same static variable order and true-first phase. It is not Lean’s `grind` and not a production SAT solver.

Instance	Variables	Clauses	DPLL decisions	CDCL decisions
Padded core, 0 unused	2	4	1	1
Padded core, 4 unused	6	4	31	5
Padded core, 8 unused	10	4	511	9
Padded core, 12 unused	14	4	8,191	13
Pigeonhole 4 $\rightarrow$ 3	12	22	8	8
Pigeonhole 5 $\rightarrow$ 4	20	45	51	39
Pigeonhole 6 $\rightarrow$ 5	30	81	374	148

The padded-core family is an artificial diagnostic. It puts unused Boolean variables before a two-variable inconsistent core. A production preprocessor would delete those unused variables. Its large decision-count gap illustrates nonchronological backjumping; it is not credible evidence of practical speedup on realistic inputs. The unsatisfiable core is

$$(a \vee b) \wedge (a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b).$$

The pigeonhole encoding requires each pigeon to choose at least one hole and forbids two pigeons sharing a hole. It needs no at-most-one-hole clause for each pigeon to establish the stated impossibility. The files include every clause and the replayable proof additions.

Runtime gives a more sober picture than decision counts: in the recorded run, scanning CDCL and chronological DPLL each took about 26 milliseconds on the largest pigeonhole case, despite their different decision counts; replay added about 21 milliseconds. The exact measurements are in `benchmarks.csv`. Single tiny-instance timings are noisy, so this article treats operation counts as diagnostic and makes no general speed claim. The experiment motivates efficient propagation and proof checking as necessary engineering, rather than proving that learning alone makes an implementation faster.

11.4 Polynomial search and checker results

Test	Target and assumptions	Search result
Quartic square	$x^4 + y^4 - 2x^2y^2 \geq 0$	Certified
Three-square quartic	$x^4 + y^4 + 1 - x^2y^2 - x^2 - y^2 \geq 0$	Certified
Cauchy–Schwarz, two terms	$(a^2 + b^2)(c^2 + d^2) - (ac + bd)^2 \geq 0$	Certified
Box bilinear	$x + y - 2xy \geq 0$ on $[0, 1]^2$	Certified
Interval product	$1 - x^2 \geq 0$, given $1 - x, 1 + x \geq 0$	Certified
Equality constrained	$x^2 + y^2 - 1/2 \geq 0$, given $x + y = 1$	Certified
Positive slack	$x^2 + y^2 + 1 - 1/2 \geq 0$	Certified

Test	Target and assumptions	Search result
False polynomial	$x^2 - 1 \geq 0$ without assumptions	Unknown
Motzkin, bounded search	$x^4y^2 + x^2y^4 + 1 - 3x^2y^2 \geq 0$, degree cap 6	Unknown
Restricted-ratio gap	$(2x + y)^2 \geq 0$, degree cap 2	Unknown

The first three identities have short decompositions:

$$\begin{aligned}
 x^4 + y^4 - 2x^2y^2 &= (x^2 - y^2)^2, \\
 x^4 + y^4 + 1 - x^2y^2 - x^2 - y^2 &= \frac{1}{2}(x^2 - y^2)^2 + \frac{1}{2}(x^2 - 1)^2 + \frac{1}{2}(y^2 - 1)^2, \\
 (a^2 + b^2)(c^2 + d^2) - (ac + bd)^2 &= (ad - bc)^2.
 \end{aligned}$$

The false-polynomial search returning unknown is correct behavior for a certificate searcher; the point $x = 0$ independently refutes the target. The Motzkin target is nonnegative by applying arithmetic–geometric mean to the three nonnegative terms x^4y^2 , x^2y^4 , and 1; the implemented bounded dictionary does not discover a certificate. This experiment alone says nothing about whether a stronger existing Lean SOS tactic succeeds on it.

For each of the seven found certificates, the test rejected an altered target and a negative-weight mutation. A further 100 randomly planted rational-square certificates exercised the checker and were cross-checked by SymPy expansion. Those are *checker tests*, not 100 additional search successes. The manually constructed discriminant certificate is also separate from the seven discovery successes.

11.5 Invariant synthesis and independent replay

The six powers $p = 0, \dots, 5$ yielded degrees $1, \dots, 6$. The final sample counts were 3, 4, 5, 6, 7, 8, respectively; increasing-degree search histories are included. Each result passed exact base and step identities. Each also rejected the alterations $Q + 1$ and $Q + n$, giving 12 synthesis-side mutation rejections.

A separate standard-library replay then reconstructed the six polynomials from rational coefficients and rechecked both residuals with sparse polynomial substitution. It rejected the same 12 mutations independently of SymPy. This is stronger than checking sampled recurrence values, but remains Python verification, not a formally proved checker implementation.

11.6 Saved-certificate replay summary

The retained corpus has eight SAT/theory proof logs, eight polynomial certificates (seven discovered and one manually supplied), and six invariant certificates. All replayed successfully under `python -S`. Three polynomial search files have no certificate and are explicitly skipped as unknowns. The replay program fails on a missing corpus rather than reporting vacuous success.

11.7 What the experiments establish

The experiments support four limited conclusions. First, proof explanations can connect Boolean learning and an exact theory without trusting the searcher. Second, a small numerical search can discover useful nonlinear certificates whose replay needs no numerical software. Third, polynomial invariant discovery

can be separated from exact inductive validation. Fourth, the chosen small implementations have visible limitations that the architecture must address.

They do not establish soundness of arbitrary Python execution, correctness of a Lean reifier, successful integration of all plugins, or superiority to `grind`. Those are distinct validation obligations, not quantities that can be inferred from passing toy tests.

12 A serious Lean evaluation protocol

12.1 Pinning and baseline retention

Record the Lean release and commit, Mathlib revision, every optional package revision, solver versions, native build flags, hardware, and trust profile. Avoid comparing an aggressively tuned Forge invocation with an artificially constrained `grind` call. Include both a default `grind` baseline and a baseline with the same available lemmas and comparable resource budgets.

The baseline-retention suite should cover dependent equalities, collections, integer arithmetic, finite-characteristic algebra, structured case analysis, and standard library lemmas. Measure cold and warm runs separately. Any proof cache or premise index that amortizes work must be counted explicitly rather than charged only to the competitor.

12.2 New-coverage families

Use fresh recursive definitions for accumulator and list identities so that theorem-name retrieval does not masquerade as induction. Hold out whole definition families, not random near-duplicate goals. Include nonlinear targets requiring side conditions, existential goals with genuinely shared witnesses, and mixed Boolean/theory problems whose non-Boolean explanations matter.

For library-heavy tasks, forbid the target theorem and declarations depending on it from the premise set. Otherwise a tactic may “prove” the goal by retrieving the already proved result. Renaming a theorem is not enough if its exact type remains indexed as a premise. Record the dependency slice used by each successful proof.

Compare with Aesop, appropriate specialized arithmetic tactics, SOS, bit-vector SAT automation, and a strong portfolio/hammer baseline where installed. These comparisons answer different questions: broader coverage than `grind`, integration value beyond a portfolio, and runtime against the most suitable specialist.

12.3 Metrics and ablations

Report solved counts, unique new solves, baseline regressions, median and tail latency, total discovery-plus-replay time, kernel checking time, peak memory, certificate bytes, proof size, and dependency count. Do not report only successes: timeouts and reconstruction failures belong in the denominator.

Run component ablations which disable induction, witness construction, learned clauses, intermediate-bound publication, nonlinear certificates, and premise selection. A particularly important comparison keeps all engines but disables fact sharing between them. That tests whether the cooperation architecture earns its complexity beyond ordinary tactic sequencing.

For stochastic or learned ranking, use fixed seeds and repeat runs; for small deterministic examples, repeat timing sufficiently to characterize noise. Establish a held-out tuning split for heuristic weights and budgets. No numerical speed or coverage target in this article has already been achieved by a Lean implementation.

13 Failure modes and regression obligations

Failure mode	Required discipline	Regression example
Branch fact escapes	Discharge decision assumptions or retain scope.	A bound proved only under $x \geq 0$ must not survive the opposite branch.
Witness depends on a later binder	Check permitted free-variable closure.	Distinguish $\forall x \exists y$ from $\exists y \forall x$.
Shared witness split accidentally	Preserve one metavariable across conjuncts.	Reject incompatible obligations on the same existential witness.
Natural subtraction treated as ring subtraction	Use carrier-specific reification/bridges.	$0 - 1 = 0$ in $\mathbb{N}$.
Integer complement encoded as real complement	Use $-c - 1$ only in integer difference logic.	Negation of $x - x \leq 0$ is inconsistent over integers.
Denominator cleared without sign	Generate nonzero/sign demands.	Inequality direction changes when the divisor is negative.
Equality cofactor constrained or mis-signed	Free ideal coefficients, nonnegative cone coefficients.	Discriminant identity (2).
Numerical residual accepted	Exact coefficient equality only.	Perturb a coefficient below solver tolerance.
Sampled invariant accepted as theorem	Prove base and step symbolically.	Add a polynomial vanishing on sampled points.
Circular synthesized lemma	Only recursor-scoped induction hypotheses.	Reusing the candidate at the same argument is forbidden.
Dependent transport erased	Native equality or explicit semantic bridge.	Vector operations at propositionally equal indices.
Certificate file transplanted	Bind certificate to exact reified problem.	Same variable count, different clause or polynomial.
Native computation hidden	Audit transitive dependencies and trust profile.	Replay in strict mode must reject added evaluation axioms.
Failure reported as counterexample	Separate unknown, error, and validated model.	Restricted square dictionary misses a true inequality.

The prototype tests several of these mechanisms, but not the dependent-type, Lean trust-profile, or metavariable cases. Those belong to the future Lean regression suite. Resource exhaustion is also a first-class outcome: certificate parsing, monomial expansion, and theorem instantiation require allocation limits before processing adversarially large data.

14 Reproduction and bundle inventory

From the archive root, a complete search/test run is

```
python -m pip install -r requirements.txt
python prototype/run_tests.py --out reproduced-results
python -S prototype/replay_certificates.py --results reproduced-results
```

The first command is only needed for discovery/tests in an environment without the recorded dependencies. Saved-certificate replay needs no third-party Python package:

```
python -S prototype/replay_certificates.py --results results
```

On systems where numerical libraries create many worker threads, setting `OPENBLAS_NUM_THREADS=1` and `OMP_NUM_THREADS=1` helps make these small runs easier to compare. The recorded run used those settings. The command-line scripts use explicit result-directory arguments so the supplied record can be preserved.

The bundle contains the article source/PDF, four algorithm/test modules plus the replay script, machine-readable certificates, generated counts and timing tables, uncompiled Lean specimens, and a README describing artifact status. The Makefile provides convenient targets. It intentionally contains no Lean binaries, third-party solver sources, or font files.

To rebuild the article, run `latexmk -pdf forge.tex` from `article/` with a LaTeX installation containing the listed packages. The document uses ordinary LaTeX packages and no shell-escape requirement. The bibliography is included directly in the source, so no bibliography database or external bibliography tool is required.

15 Recommendation

The most promising implementation is a layered Forge whose first useful release adds *induction with generalization*, *typed witness construction*, and *intermediate certified arithmetic facts* to the native `grind` foundation. Add tightly integrated CDCL(T) after the scope and explanation contracts are reliable. This ordering produces new capability before undertaking the deepest core surgery.

The technical ingredients are concrete. The ground layer learns proof-backed clauses; the nonlinear layer returns exact cone identities; the induction layer proposes generalized statements and checks recursor obligations; the witness layer constructs terms in their true binder scope; and the demand manager sends each unresolved obligation to the engine best suited to it. Existing Lean projects already supply important parts, so the engineering problem is disciplined integration rather than inventing every solver.

The supplied prototypes make three parts executable and falsifiable. They also expose why the remaining work matters: search dictionaries can miss trivial truths, learning can reduce decisions without reducing runtime, and exact Python replay is not a substitute for a Lean proof bridge. A successful Forge should make all these boundaries visible while allowing its components to cooperate much more effectively than a list of tactic fallbacks.

A Certificate formats and implementation details

A.1 SAT/theory JSON

A retained SAT file contains n , original clauses, a map of atoms, and a sequential proof. An integer literal is nonzero and its absolute value is at most n . An atom stores integer x , y , and c , denoting $x - y \leq c$.

```
{
  "kind": "theory", "clause": [-1, -2, -3],
  "cycle": [1, 2, 3]
}
{"kind": "rup", "clause": []}
```

This two-event example is valid when the original clauses assert all three atoms of the negative-cycle example. General logs interleave theory and RUP additions. A replayed empty clause must be the final event; truncation is rejected. The current format does not use deletion or explicit RUP hint chains.

A.2 Polynomial JSON

A polynomial has a variable count and sparse entries `[exponentVector, rationalString]`. Duplicate monomials are rejected on decoding. A cone entry has a nonnegative rational `weight`, a polynomial `square`, and a list of assumption indices `factors`; the name `square` stores q , not q^2 . An ideal entry has a `hypothesis` index and a polynomial `multiplier`.

The target and assumptions are stored alongside the certificate. Verification expands the right side and compares its exact coefficient map with the target. There is no epsilon acceptance criterion. Malformed dimensions, invalid indices, and negative weights fail the check. This is a mathematical prototype format, not yet a hardened untrusted-input protocol with comprehensive size quotas.

A.3 Invariant JSON

The file records the power p , coefficients of $Q(n, a)$, search history, and diagnostic residual strings. The replay ignores the diagnostic residual claims: it independently reconstructs Q , substitutes into (4)–(5), and compares coefficient maps with zero. This distinction prevents a fabricated “residual: 0” field from serving as evidence.

A.4 Testing scope and remaining implementation limitations

The SAT implementation scans clauses, has no restart or deletion policy, and rebuilds a small theory graph for consistency checks. The polynomial implementation uses naive sparse multiplication and simple support reconstruction. The invariant implementation only covers the supplied accumulator family and polynomial ansatz. None is optimized for large problems.

The independent polynomial replay shares the sparse polynomial data type with the search-side checker, while search itself uses numerical LP proposals. Invariant replay uses that sparse data type instead of SymPy, providing a stronger separation from the synthesis checker. These are deliberate engineering checks, not formal proofs of the Python implementations.

B Lean integration specimens

The file `lean/Examples.lean` contains ordinary theorem proofs illustrating the induction and arithmetic shapes. It does not define `forge`. The file `lean/Contracts.lean` sketches a small proof-carrying result interface using Lean structures; it is not a complete plugin API. Both are clearly marked uncompiled. The source is intended to be checked and adjusted after selecting a compatible Lean/Mathlib manifest.

The important algorithmic decision is the motive, not the final normalization tactic. A syntactically valid call to induction with a fixed accumulator gives an induction hypothesis too weak for the recursive call. The mathematical induction skeleton for the triangular accumulator is:

```
-- Specimen, not executed in this environment.
-- Introduce n, then generalize the changing accumulator a.
induction n generalizing a with
| zero => simplify the defining equation
| succ n ih =>
  unfold one recursive equation
  specialize ih at the updated accumulator
  normalize the resulting polynomial identity
```

References

- [1] Lean contributors. *The Lean Language Reference: The grind tactic*. Rolling reference, accessed 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean contributors. *Algebraic Solver (Commutative Rings, Fields)*. Accessed 14 September 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-_LPAR_Commutative-Rings___-Fields_RPAR_/.
- [3] Lean contributors. *Linear Integer Arithmetic*. Accessed 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Integer-Arithmetic/>.
- [4] Lean contributors. *Linear Arithmetic Solver*. Accessed 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Arithmetic-Solver/>.
- [5] Kim Morrison and Leonardo de Moura. *grind: An SMT-Inspired Tactic for Lean 4 (Short Paper—System Description)*. IJCAR 2026, LNCS 16688, pp. 106–114. Author’s HTML version: <https://kim-em.github.io/papers/grind/>. DOI: https://doi.org/10.1007/978-3-032-32589-1_7.
- [6] Lean contributors. *Lean v4.34.0 release*. GitHub release endpoint inspected 14 September 2026; publication timestamp 2026-09-14T14:04:36Z. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [7] Lean contributors. *Lean/Meta/Tactic/Grind/Main.lean*, tag v4.34.0. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Main.lean>.
- [8] Lean contributors. *Lean/Meta/Tactic/Grind/Types.lean*, tag v4.34.0. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Types.lean>.
- [9] Lean contributors. *E-matching*. Accessed 14 September 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/E___matching/.
- [10] Aesop contributors. *Aesop: Proof search for Lean*. Repository documentation, accessed 14 September 2026. <https://github.com/leanprover-community/aesop>.
- [11] Lean SOS contributors. *SOS: Sum-of-squares nonlinear arithmetic in Lean 4*. Current repository architecture and API documentation, accessed 14 September 2026. <https://github.com/leanprover/sos>.
- [12] Lean LP contributors. *Linear programming in Lean 4*. Current repository package, backend, tactic, and trust-model documentation, accessed 14 September 2026. <https://github.com/leanprover/lp>.
- [13] Kim Morrison. *Verified linear programming*, 14 June 2026. Author’s technical overview. <https://kim-em.github.io/blog/2026-6-14-verified-linear-programming/>.
- [14] Lean contributors. *Lean.Elab.Tactic.BVDecide*. Generated source API documentation. Accessed 14 September 2026. https://leanprover-community.github.io/mathlib4_docs/Lean/Elab/Tactic/BVDecide.html.
- [15] Lean-SMT authors. *Lean-SMT: An SMT tactic for discharging proof goals in Lean*. CAV 2025 / arXiv:2505.15796. <https://arxiv.org/abs/2505.15796>. Implementation: <https://github.com/ufmg-smite/lean-smt>.
- [16] Duper contributors. *Duper: Automated theorem proving for Lean*. Repository, accessed 14 September 2026. <https://github.com/leanprover-community/duper>.
- [17] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. *Lean-Auto: An Interface Between Lean 4 and Automated Theorem Provers*. CAV 2025. Implementation: <https://github.com/leanprover-community/lean-auto>.
- [18] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. *Premise Selection for a Lean Hammer*. arXiv:2506.07477, 2025; subsequent versions available at <https://arxiv.org/abs/2506.07477>.

- [19] Marcus Rossel and contributors. *lean-egg*. Repository marked deprecated when accessed on 14 September 2026. <https://github.com/marcusrossel/lean-egg>.
- [20] Marcus Rossel and Andrés Goens. *Bridging Syntax and Semantics of Lean Expressions in E-Graphs*. arXiv:2405.10188, 2024. <https://arxiv.org/abs/2405.10188>.
- [21] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. *Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T)*. Journal of the ACM 53(6), 937–977, 2006. <https://doi.org/10.1145/1217856.1217859>.
- [22] John Harrison. *Verifying Nonlinear Real Formulas Via Sums of Squares*. TPHOLs 2007, LNCS 4732. https://doi.org/10.1007/978-3-540-74591-4_9.
- [23] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley; Technical Report UCB/EECS-2008-177, 2008. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html>.
- [24] Stefan Szeider. *PBLean: Pseudo-Boolean Proof Certificates for Lean 4*. arXiv:2602.08692, 2026, revised 2 April 2026. <https://arxiv.org/abs/2602.08692>.
- [25] Lean contributors. *RFC: One axiom per native computation*, issue 12216, opened 28 January 2026. Historical design discussion; exact current behavior must be audited in the pinned implementation. <https://github.com/leanprover/lean4/issues/12216>.